

Informe de Código Java — TP2

Explicación detallada de las partes importantes de cada programa

Nota general: Los programas Java implementan **exactamente el mismo protocolo** que los de C, por lo que son **interoperables** entre sí (podés correr el servidor en C y el cliente en Java, o viceversa). Este informe se enfoca en las **diferencias con C** y en las partes que Java maneja de forma distinta.

1. ServerPingPong.java — Servidor eco completo (Punto 3)

¿Qué hace este programa?

Lo mismo que `server_pingpong.c` : recibe N bytes de un cliente y los reenvía de vuelta. Es el “espejo” para que el cliente mida el RTT.

Partes importantes

Lectura del tamaño con `DataInputStream`

```
DataInputStream din = new DataInputStream(in);
int total = din.readInt();
```

¿Qué hace? Lee los 4 bytes de anuncio de tamaño y los interpreta como un `int` en big-endian (network byte order).

Diferencia con C: En C hay que llamar a `ntohl()` manualmente para convertir de network byte order a host byte order. En Java, `DataInputStream.readInt()` **siempre lee en big-endian** (que resulta ser el mismo que network byte order), así que la conversión es automática. Esta es una ventaja de Java: no hay que preocuparse por el endianness de la máquina.

Lectura completa con `offset`

```
byte[] buffer = new byte[total];
int received = 0;
while (received < total) {
```

```
int n = in.read(buffer, received, total - received);
if (n <= 0) break;
received += n;
}
```

¿Qué hace? Lee bytes del stream hasta completar los N bytes, acumulándolos en el buffer. El método `in.read(buffer, offset, length)` lee **hasta** `length` bytes y los coloca a partir de la posición `offset` del arreglo.

¿Por qué no usar `in.read(buffer)` ? Porque `InputStream.read(byte[])` puede devolver menos bytes de los que caben en el arreglo. Igual que el `read()` de C, **nunca garantiza leer todo**. El `while` es obligatorio. La variante con 3 argumentos (`buffer, received, total - received`) permite ir llenando el arreglo desde donde se quedó la última lectura.

Diferencia con C: En C se usa aritmética de punteros (`buffer + received`). En Java se usa el parámetro `offset` del método `read()` . El efecto es el mismo.

Reenvío con write

```
out.write(buffer, 0, received);
out.flush();
```

¿Qué hace? Escribe todos los bytes recibidos de vuelta al cliente.

Diferencia crucial con C: En C, `write()` puede hacer una **escritura parcial** (devolver menos bytes de los pedidos), así que se necesita un `while` . En Java, `OutputStream.write(byte[], offset, length)` **bloquea hasta escribir todo el contenido**. No existe el concepto de escritura parcial visible al programador — la JVM lo maneja internamente. Por eso **no hace falta un bucle** alrededor del `write()` en Java.

¿Por qué el `flush()` ? Porque `OutputStream` puede tener un buffer interno (especialmente si está envuelto en un `BufferedOutputStream`). `flush()` fuerza a que los datos pasen al buffer del socket del kernel para ser enviados por TCP. Sin `flush()` , los datos podrían quedar en memoria de la JVM sin enviarse.

2. ClientPingPong.java — Cliente con medición de RTT (Punto 3)

¿Qué hace este programa?

Envía N bytes al servidor, espera el eco completo, mide el RTT y calcula RTT/2.

Partes importantes

Medición con `System.nanoTime()`

```
long t0 = System.nanoTime();
out.write(data);
out.flush();
// ... read loop ...
long t1 = System.nanoTime();
double rttUs = (t1 - t0) / 1000.0;
```

¿Qué hace? `System.nanoTime()` devuelve el tiempo del reloj monótono en nanosegundos. Se divide por 1000 para convertir a microsegundos.

Equivalencia con C: Es el equivalente directo de `clock_gettime(CLOCK_MONOTONIC, ...)`. Ambos son relojes monótonos — nunca retroceden ni son afectados por cambios de hora del sistema (NTP, cambio manual, etc.). La diferencia es que en C se obtiene una estructura `timespec` (segundos + nanosegundos) y hay que calcular la diferencia manualmente, mientras que en Java se obtiene directamente un `long` en nanosegundos, lo que simplifica la aritmética.

¿Por qué no `System.currentTimeMillis()`? Porque `currentTimeMillis()` usa el reloj de pared del sistema, que puede ser ajustado por NTP durante la medición. Además, su resolución es de milisegundos, insuficiente para medir operaciones que tardan microsegundos.

El write en Java no necesita bucle

```
out.write(data);
out.flush();
```

Comparación directa con C:

```
// En C, esto es necesario:
while (sent < total) {
    ssize_t n = write(sockfd, data + sent, total - sent);
    sent += (uint32_t)n;
}
```

¿Por qué en Java no hace falta? Como se explicó arriba, `OutputStream.write(byte[])` en Java garantiza internamente escribir el arreglo completo, bloqueando si es necesario. La JVM maneja las escrituras parciales por vos. En C, el programador debe encargarse manualmente.

Implicancia para la medición: En Java, el tiempo medido para `write()` incluye todo el tiempo que la JVM tardó internamente en completar escrituras parciales (si las hubo). En C, cada escritura parcial se mide por separado. Para el propósito de medir el tiempo total, el resultado es equivalente.

3. ClientSleepTest.java — Experimento de desfase temporal (Punto 5)

¿Qué hace este programa?

Lo mismo que `client_sleep_test.c` : permite insertar una pausa de N segundos en 4 puntos distintos del ciclo de conexión/envío/recepción.

Partes importantes

El sleep en Java vs C

```
Thread.sleep(sleepSecs * 1000L); // Java: milisegundos
```

```
sleep(sleep_secs); // C: segundos
```

Diferencia: `Thread.sleep()` recibe milisegundos, por eso se multiplica por 1000. `sleep()` de C recibe segundos directamente. La `L` en `1000L` fuerza aritmética `long` para evitar overflow si `sleepSecs` fuera grande (no es el caso acá, pero es buena práctica).

¿Por qué `Thread.sleep()` y no otra cosa? Java no tiene un equivalente directo del `sleep()` de POSIX. `Thread.sleep()` es la forma estándar de pausar el hilo actual. También existe `TimeUnit.SECONDS.sleep(n)` (más legible), pero `Thread.sleep()` es más tradicional y directo.

El connect en Java es implícito

```
/* MODO 1: Sleep ANTES del connect */
if (modo == 1) {
    Thread.sleep(sleepSecs * 1000L);
}
Socket socket = new Socket(ip, port); // connect() implícito acá
```

Diferencia con C:

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0); // Solo crea el socket
// ... se puede dormir acá sin problemas ...
connect(sockfd, ...); // Connect explícito, separado
```

¿Por qué importa? En C, `socket()` y `connect()` son dos llamadas separadas. Se puede crear el socket, dormir 10 segundos, y luego conectar. En Java, `new Socket(ip, port)` **crea el socket Y conecta en una sola operación**. No hay forma de separarlas con el constructor estándar.

¿Se podría separar en Java? Sí, usando el constructor vacío + `connect()` :

```
Socket socket = new Socket();           // Solo crea
socket.connect(new InetSocketAddress(ip, port)); // Conecta después
```

Pero para este experimento no es necesario — lo que importa es que el sleep ocurra antes de que exista la conexión, y eso se logra poniendo el `Thread.sleep()` antes del `new Socket(ip, port)` .

Mediciones separadas por fase

```
long tc0 = System.nanoTime();
Socket socket = new Socket(ip, port);
long tc1 = System.nanoTime();
double tConnect = (tc1 - tc0) / 1000.0;

long tw0 = System.nanoTime();
out.write(data);
out.flush();
long tw1 = System.nanoTime();
double tWrite = (tw1 - tw0) / 1000.0;

long tr0 = System.nanoTime();
while (received < total) { ... in.read(...) ... }
long tr1 = System.nanoTime();
double tRead = (tr1 - tr0) / 1000.0;
```

¿Qué hace? Mide cada fase (connect, write, read) **por separado** con su propio par de timestamps. En C se hace exactamente lo mismo con `clock_gettime()` .

¿Por qué es importante? Permite ver **cuál fase específica** cambia con cada modo de sleep. Por ejemplo, en el Modo 3 (sleep después de write, antes de read), se espera que `tRead` sea significativamente **menor** que en el baseline, porque los datos del eco ya están en el buffer del socket del kernel cuando el `read()` se ejecuta.

4. ServerTimed.java — Servidor con medición de tiempos (Punto 2)

Partes importantes

Buffer fijo de 255 bytes para read

```
static final int READ_CHUNK = 255;
byte[] buffer = new byte[READ_CHUNK];
int n = in.read(buffer);
```

¿Qué hace? Lee hasta 255 bytes por llamada. Es el mismo tamaño que en C (`#define READ_CHUNK 255`).

Diferencia sutil con C: En C, `read(connfd, buffer, 255)` puede leer **hasta** 255 bytes. En Java, `in.read(buffer)` puede leer **hasta** `buffer.length` bytes (que es 255). El comportamiento es idéntico: ambos devuelven el número de bytes efectivamente leídos, que puede ser desde 1 hasta 255.

CSV con formato idéntico al de C

```
csv.printf("%d,%d,%d,%.3f\n", total, call, n, (t1 - t0) / 1000.0);
```

¿Por qué el mismo formato? Porque el script de graficación `plot_times_tp2.py` lee CSVs con el formato `bytes_totales,nro_llamada,bytes_transferidos,tiempo_us` . Si C y Java generan el mismo formato, se puede usar el mismo script para graficar ambos, sin importar en qué lenguaje se corrió el experimento.

5. ClientTimed.java — Cliente con medición de tiempos (Punto 2)

Partes importantes

Una sola llamada a write (sin bucle)

```
long t0 = System.nanoTime();
out.write(data);
out.flush();
long t1 = System.nanoTime();
csv.printf("%d,1,%d,%.3f\n", total, total, (t1 - t0) / 1000.0);
```

¿Por qué se reporta siempre “1” llamada? En Java, `OutputStream.write(byte[])` **siempre escribe el arreglo completo**. No existe la posibilidad de escritura parcial visible al programador. Por eso el número de llamadas es siempre 1 y los bytes escritos son siempre `total` .

Diferencia con C: En C, `write()` puede devolver menos bytes, y el CSV puede tener múltiples filas para el mismo tamaño total (una por cada llamada parcial). En Java siempre hay exactamente una fila por tamaño.

¿Esto afecta la comparación? No significativamente. Lo que importa es el **tiempo total** de enviar N bytes, que es la suma de todas las llamadas en C o la única llamada en Java. El script de graficación suma los tiempos por tamaño total, así que el resultado final es comparable.

Resumen de diferencias clave C vs Java

Aspecto	C	Java
Escritura parcial	<code>write()</code> puede devolver menos necesita <code>while</code>	<code>OutputStream.write()</code> completa todo internamente
Network byte order	Conversión manual con <code>htonl()</code> / <code>ntohl()</code>	<code>DataOutputStream/DataInputStream</code> usa big-endian automáticamente
Reloj monótono	<code>clock_gettime(CLOCK_MONOTONIC)</code> <code>struct timespec</code>	<code>System.nanoTime()</code> <code>long</code> en nanosegundos
Sleep	<code>sleep(segundos)</code>	<code>Thread.sleep(milisegundos)</code>
Connect	<code>socket()</code> + <code>connect()</code> separados	<code>new Socket(ip, port)</code> los combina
Memoria	<code>malloc()</code> / <code>free()</code> manual	<code>new byte[]</code> , el garbage collector libera
Buffer del socket	Acceso directo via syscalls	Abstracción via <code>InputStream</code> / <code>OutputStream</code>
Interoperabilidad	Mismo protocolo (4 bytes + N bytes)	Mismo protocolo, interoperable con C